

## ➤ Clases bases de POO

- **Abstracción:** Principio fundamental que permite representar objetos del mundo real o conceptos complejos a código mostrando información esencial y ocultando detalles innecesarios.

Ejemplo.

Cochete: Una clase coche puede tener atributos como "color", "marca", "modelo" y métodos como "acelerar" y "frenar". Y para esto no es necesario que el código detalle cómo funciona el motor.

- **Clase en C++:** Tipo de dato definido por el usuario que encapsula datos y funciones sobre estos datos. A los datos se les conoce como atributos y a las funciones como métodos. La clase es una plantilla o modelo para crear objetos y a estructurar el código.

Sus principales características son:

- **Encapsulamiento:** Las clases combinan atributos y métodos en una sola unidad, ocultando detalles internos y exponiendo una sola interfaz.
- **Abstracción:** Permite la creación de datos abstractos donde define un objeto sin entrar en detalles de la implementación.
- **Herencia:** Entre clases pueden heredarse propiedades y métodos, lo que permite reutilizar código y crear jerarquías entre ellas.
- **Polimorfismo:** Las clases pueden comportarse de manera diferente según el tipo de objeto.

- **Método:** Es una función dentro de una clase y define el comportamiento de los objetos de esa clase. Encapsulan el comportamiento de objetos y permite que interactúen con sus datos y realicen diversas tareas.

Ejemplo.

El método "Asistir-clase" para la clase alumno, define el comportamiento.



- **Atributo:** Son las características y propiedades que definen a un objeto. Variables que almacenan datos sobre el objeto y determinan su estado.

Ejemplo:

Para la clase puerta, pueden existir los atributos de "color", "tamaño", "tipo".

- **Niveles de visibilidad y tipos de acceso:**

Existen tres tipos de niveles de visibilidad:

- **Public:** recomendado para los métodos principales de una clase, es visible para todas las instancias creadas del objeto en el programa.

- **Private:** suele utilizarse para los atributos, restringe el acceso para los métodos fuera de la clase donde provienen previniendo fuga de datos.

- **Protected:** al igual que 'private' restringe el acceso a miembros externos, su principal diferencia es que le da acceso solo a clases que se relacionan por medio de la herencia.

- **Ejemplo de una clase:**

class Gato {

private:

String raza;

char[50] nombre;

int edad;

String color;

public:

void Ronronear();

void Maullar();

void Cazar-ratones();

};